

Поглиблено. Автоматизація повсякденних фронтендзадач з ШІ

№ уроку: 2 Курс: FrontEnd.AI Skills

Ціль уроку

Як ШІ допомагає у фронтенд розробці на HTML, CSS, JavaScript.
Написання тестів до JavaScript функцій за допомогою ШІ.
Автоматична генерація документації та коментарів до HTML/CSS/JS коду.
Пояснення чужого коду за допомогою ШІ.
Генерація шаблонів сторінок, компонентів інтерфейсу і повторюваних фрагментів коду.
Недоліки та ризики генерації коду ШІ.
Огляд безкоштовних і платних інструментів ШІ, які корисні для фронтенд розробника.

Зміст уроку:

1. Як може бути корисним ШІ в програмуванні (на прикладі HTML, CSS, JS).
2. Як ШІ допомагає писати тести для JavaScript функцій.
3. Які можливості має генерація тестів.
4. Які недоліки у згенерованих тестах.
5. Як автоматично генерувати документацію до коду (коментарі, README, пояснення).
6. Як пояснювати чужий HTML/CSS/JS код або код з реального проєкту.
7. Використання ШІ для створення шаблонів сторінок, блоків, скриптів.
8. Які складнощі можуть бути при генерації коду.
9. Чи залежить якість генерації від мови програмування.
10. Чи можна користуватися ШІ безкоштовно та яка різниця з платними версіями.

Резюме

Урок присвячений практичному використанню ШІ в роботі фронтенд-розробника з HTML, CSS та JavaScript. Студенти дізнаються, як ШІ може пришвидшувати написання коду (генерація розмітки, стилів, скриптів), допомагати працювати з DOM, валідацією форм, подіями, а також пояснювати помилки з консолі браузера й підказувати варіанти їх виправлення. Окремо розглядається, як за допомогою ШІ виконувати рефакторинг: спрощувати довгі функції, покращувати назви змінних, структурувати скрипти, виносити стилі в окремі файли та створювати базову документацію (коментарі в коді, README.md).

Урок також показує, як ШІ допомагає в тестуванні JavaScript: генерація unit-тестів, продумування кейсів (нормальних, граничних, некоректних значень), створення мок-даних та використання підходу Arrange-Act-Assert. Окремий блок присвячений тому, як за допомогою ШІ пояснювати чужий HTML/CSS/JS код, ділити його на логічні частини та розуміти контекст. Студенти вчаться формувати промпти для генерації шаблонів сторінок, блоків і типових скриптів (модалки, форми, меню, таби), а також розбирають типові складнощі: потребу в контексті, ручній перевірці коду, чітких технічних вимогах.

Наприкінці уроку розглядається різниця між безкоштовними та платними інструментами ШІ (ChatGPT, Gemini, Claude, Copilot, Codeium тощо), їхні обмеження та переваги для розробника. Дається набір рекомендацій, як максимально ефективно використовувати навіть безкоштовні версії: писати точні промпти, дробити задачі, завжди тестувати код у браузері та зберігати вдалі промпти як робочі шаблони. Результат — розуміння того, як зробити ШІ щоденним помічником у верстці, JS-логіці, тестуванні та документуванні проєктів.

Питання для самоперевірки

- Які задачі може вирішувати ШІ?
- Що саме ШІ робить в тестуванні?
- Які можливості в генерації тестів?
- Які недоліки згенерований тестів?
- Які є способи згенерувати документацію?
- Як правильно отримати пояснення чужого коду?
- Які шаблони може генерувати ШІ?
- Які можуть виникнути проблеми під час генерації коду?
- Як вибір мови програмування впливає на якість генерації коду?
- Яка різниця між безкоштовними та платними ШІ?

Домашнє завдання

Перейдіть на сайт ChatGPT:

<https://chatgpt.com/>

Створіть новий простий JavaScript-проект з шаблону:

створіть папку, наприклад `js-tests-project`;

всередині створіть файли:

- `index.html`
- `script.js`
- `tests.js`

Додайте будь-який JavaScript-код у `script.js` (або згенеруйте його за допомогою ШІ).

Наприклад, функцію, яка працює з масивами, рядками або числами.

Згенеруйте тести для цього коду, використовуючи промпт до ШІ, наприклад:

"Напиши юніт-тести для цього коду на чистому JavaScript, використовуючи `console.assert`, без фреймворків. Ось мій код: ..."

Створіть у файлі `tests.js` «тестовий модуль»:

додайте згенеровані тести з ChatGPT у `tests.js`;

переконайтесь, що тести викликають функції з `script.js` (через глобальні змінні/функції).

Підключіть файли у `index.html`:

```
<script src="script.js"></script>
```

```
<script src="tests.js"></script>
```

Відкрийте `index.html` в браузері та відкрийте консоль розробника (F12 → Console):

якщо є синтаксичні помилки – виправте їх самостійно

або напишіть новий промпт до ШІ, додавши текст помилки і рядок, де вона виникає;

повторно оновіть сторінку й перевірте, що всі тести відпрацьовують без помилок.

Переконайтесь, що тести дійсно перевіряють логіку:

тимчасово змініть реалізацію однієї функції так, щоб вона повертала неправильний результат; перезавантажте сторінку і подивіться, який тест «падає» в консолі.

По черзі «пройдіться» кожним тестом:

подивіться, які вхідні дані він передає у функцію; який результат очікується;

що станеться, якщо змінити вхідні дані (перевірте це вручну в консолі).

Додаткові ресурси

Загальне про ChatGPT

- <https://help.openai.com/en/collections/3742473-chatgpt>

Prompt engineering

- <https://platform.openai.com/docs/guides/prompt-engineering>
- <https://github.com/dair-ai/Prompt-Engineering-Guide>
- <https://learnprompting.org/docs/introduction>

HTML/CSS/JS (довідка)

- <https://developer.mozilla.org/uk/docs/Web/HTML>
- <https://developer.mozilla.org/uk/docs/Web/CSS>
- <https://developer.mozilla.org/uk/docs/Web/JavaScript>